Sistema de Gestión Ecommerce

Manual Técnico y Documentación del Proyecto

Autor: Samuel Nathaniel Guasco

Carrera: Ingeniería en Software

Universidad: Universidad Internacional del Ecuador

Fecha: 01-03-2026

Made with GAMMA

Introducción

Contexto Académico

Este proyecto se desarrolla como parte del plan de estudios de Ingeniería en Software, aplicando conceptos fundamentales de diseño de software, arquitectura de sistemas y bases de datos.

Propósito del Sistema

El Sistema de Gestión Ecommerce proporciona una solución integral para administrar el ciclo completo de operaciones comerciales en línea, desde el control de usuarios hasta el procesamiento de pedidos y generación de métricas de negocio.

El objetivo general del sistema es demostrar la implementación práctica de una arquitectura API REST utilizando Go como lenguaje principal, MySQL como sistema gestor de bases de datos, y HTML para la interfaz de usuario. Este proyecto cumple con los requisitos de un sistema de gestión empresarial moderno, aplicando principios de diseño modular y separación de capas.

La solución desarrollada permite a los administradores gestionar eficientemente múltiples aspectos del negocio ecommerce, incluyendo el mantenimiento de catálogos de productos, procesamiento de transacciones, control de inventario y análisis de datos comerciales.

Made with GAMMA

Descripción General del Sistema

Gestión de Usuarios

Alta, baja, modificación y consulta de usuarios del sistema con control de credenciales y permisos.

Gestión de Productos

Mantenimiento del catálogo de productos incluyendo precios, descripciones, categorías e inventario disponible.

Gestión de Pedidos

Procesamiento completo de pedidos desde la creación hasta el cálculo automático de totales y actualización de stock.

Generación de Resúmenes

Creación de métricas y estadísticas comerciales para análisis de rendimiento del negocio.

El sistema resuelve el problema de la gestión manual y dispersa de operaciones ecommerce, centralizando todas las funcionalidades en una única plataforma coherente. Al automatizar procesos como el cálculo de totales, la verificación de inventario y la generación de reportes, se reduce significativamente el margen de error humano y se optimiza el tiempo dedicado a tareas administrativas.

La arquitectura del sistema está diseñada siguiendo patrones de diseño modernos que garantizan escalabilidad, mantenibilidad y seguridad en las operaciones comerciales.

Made with GAMMA

Arquitectura del Sistema

El sistema sigue una arquitectura de capas claramente definida, separando las responsabilidades del frontend y backend mediante una API REST. Esta separación permite una evolución independiente de cada capa y facilita el mantenimiento del código.

Capa de Presentación

Interfaz HTML que comunica con el backend mediante peticiones HTTP

Capa de API REST

Endpoints HTTP que procesan solicitudes y generan respuestas en formato JSON

Capa de Datos

MySQL gestiona la persistencia y relaciones entre entidades del sistema

La estructura del proyecto sigue convenciones estándar de Go:

- **cmd/api/**: Contiene el servidor principal y el punto de entrada del programa
- **models.go**: Define las estructuras de datos Go que mapean a tablas de MySQL
- **handlers.go**: Implementa la lógica de negocio y procesamiento de requests HTTP
- **queries.go**: Contiene todas las consultas SQL parametrizadas del sistema
- **database.go**: Gestiona la conexión a MySQL y la inicialización del esquema

Made with GAMMA

Diagrama de Flujo del Sistema

Nota: Inserte el diagrama de flujo proporcionado en esta sección. El diagrama debe mostrar el ciclo principal del sistema, las opciones del menú, las operaciones de gestión (usuarios, productos, pedidos, resumen) y la ruta de salida del programa.

Estructura del Flujo Principal

El sistema ejecuta un bucle continuo que presenta al usuario un menú principal con cinco opciones de funcionalidad. Cada opción del menú dirige al sistema hacia una rutina específica de gestión o consulta.

01

Presentación del Menú

Visualización de opciones disponibles al usuario administrador

02

Lectura de Opción

Captura de selección del usuario mediante entrada estándar

03

Procesamiento de Selección

Ejecución del caso de uso correspondiente a la opción elegida

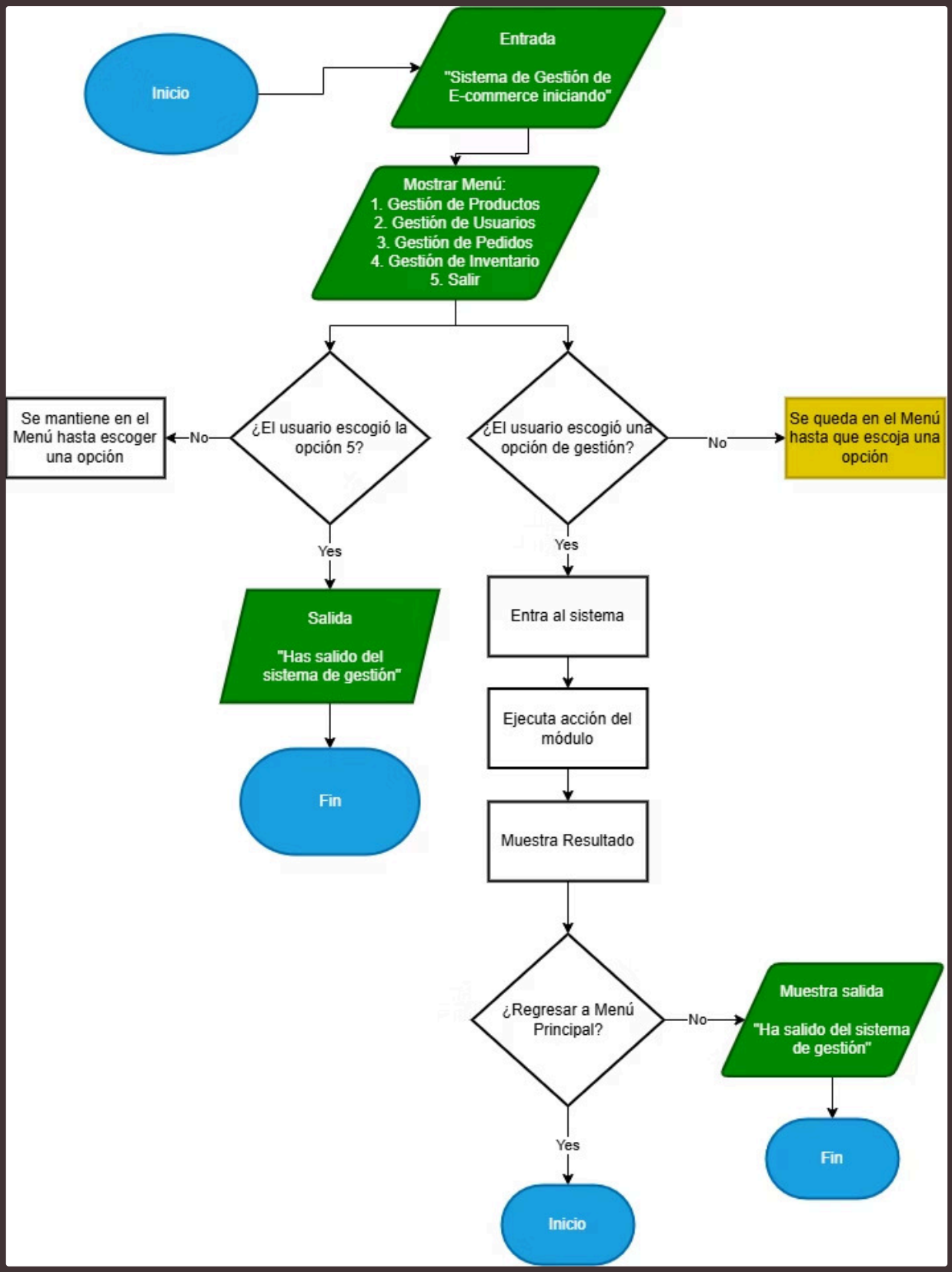
04

Retorno al Menú

Ciclo continúa hasta que se selecciona la opción de salida

El flujo incluye validaciones de entrada para garantizar que el usuario seleccione opciones válidas. Las operaciones de gestión permiten crear, consultar, modificar y eliminar registros en las diferentes entidades del sistema.

Made with GAMMA



Made with GAMMA

Diagrama de Casos de Uso

Nota: Inserte el diagrama UML de casos de uso proporcionado. El diagrama debe incluir el actor Administrador conectado a los casos de uso: Gestionar Usuarios, Gestionar Productos, Gestionar Pedidos y Generar Resumen, mostrando relaciones <> donde corresponda.

Actor del Sistema

El único actor identificado es el **Administrador**, quien tiene permisos completos para ejecutar todas las operaciones del sistema. Este actor representa a los usuarios con credenciales de acceso que pueden realizar modificaciones en la base de datos.

Gestionar Usuarios

Permite crear nuevos usuarios, consultar listados, modificar información existente y eliminar cuentas del sistema. Incluye validación de credenciales.

Gestionar Productos

Control completo del catálogo de productos: alta de nuevos ítems, actualización de precios y descripciones, consulta de stock disponible y eliminación de productos.

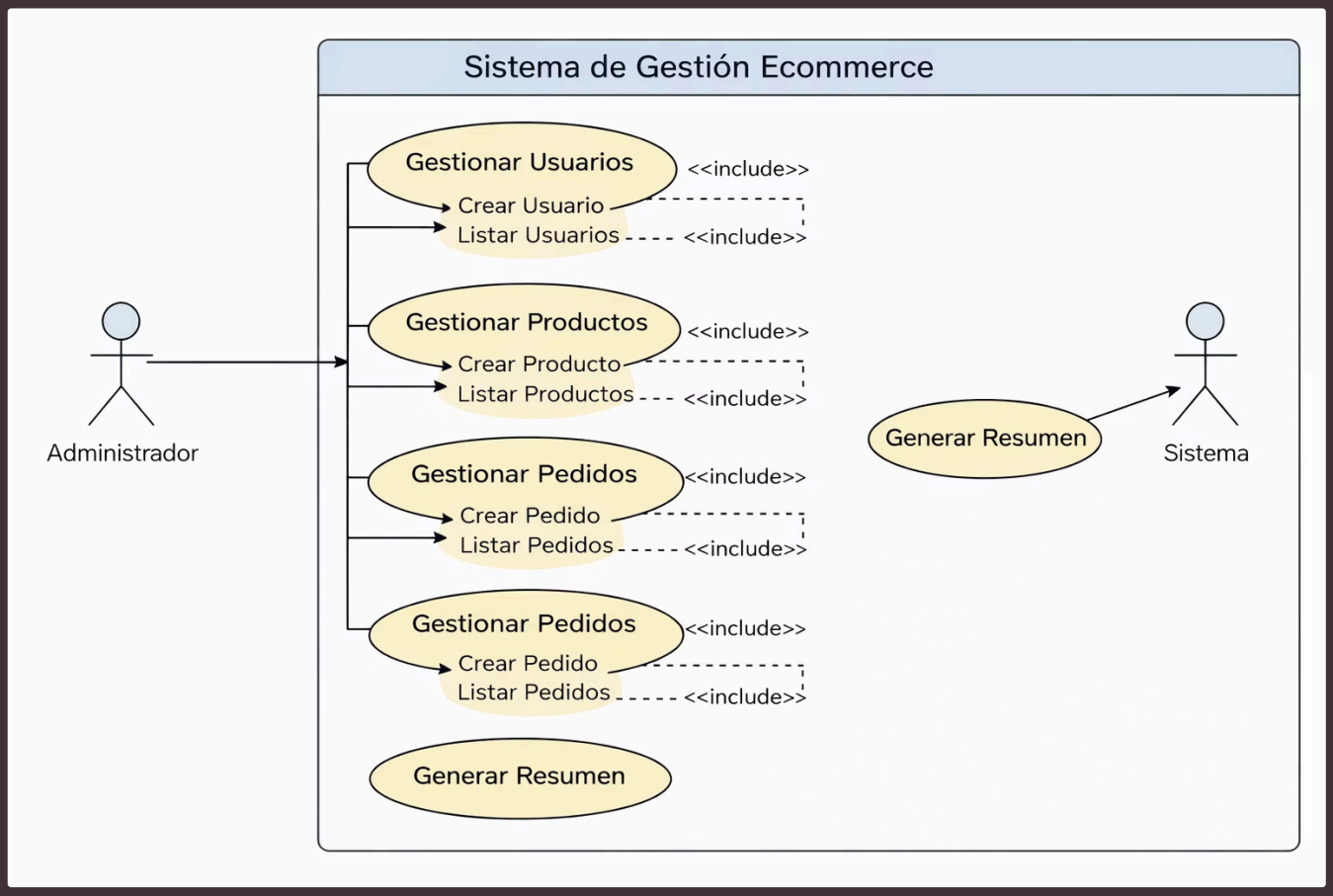
Gestionar Pedidos

Procesamiento de transacciones comerciales: creación de nuevos pedidos vinculados a usuarios y productos, cálculo automático de totales y mantenimiento del historial.

Generar Resumen

Consulta analítica del sistema mostrando métricas como ingresos totales, cantidad de pedidos procesados, productos más vendidos y estadísticas de usuarios activos.

Made with GAMMA



Made with GAMMA

Diagrama UML de Clases

Nota: Inserte el diagrama UML de clases proporcionado. El diagrama debe mostrar las clases User, Product, Order y OrderItem con sus atributos, métodos y relaciones de asociación, incluyendo multiplicidades.

Clase User

Representa a los usuarios del sistema con atributos: ID, nombre, email, contraseña y fecha de registro. Métodos de acceso y validación de credenciales.

Clase Product

Modela los productos disponibles para venta con: ID, nombre, descripción, precio unitario, stock disponible y categoría. Incluye métodos de actualización de inventario.

Clase Order

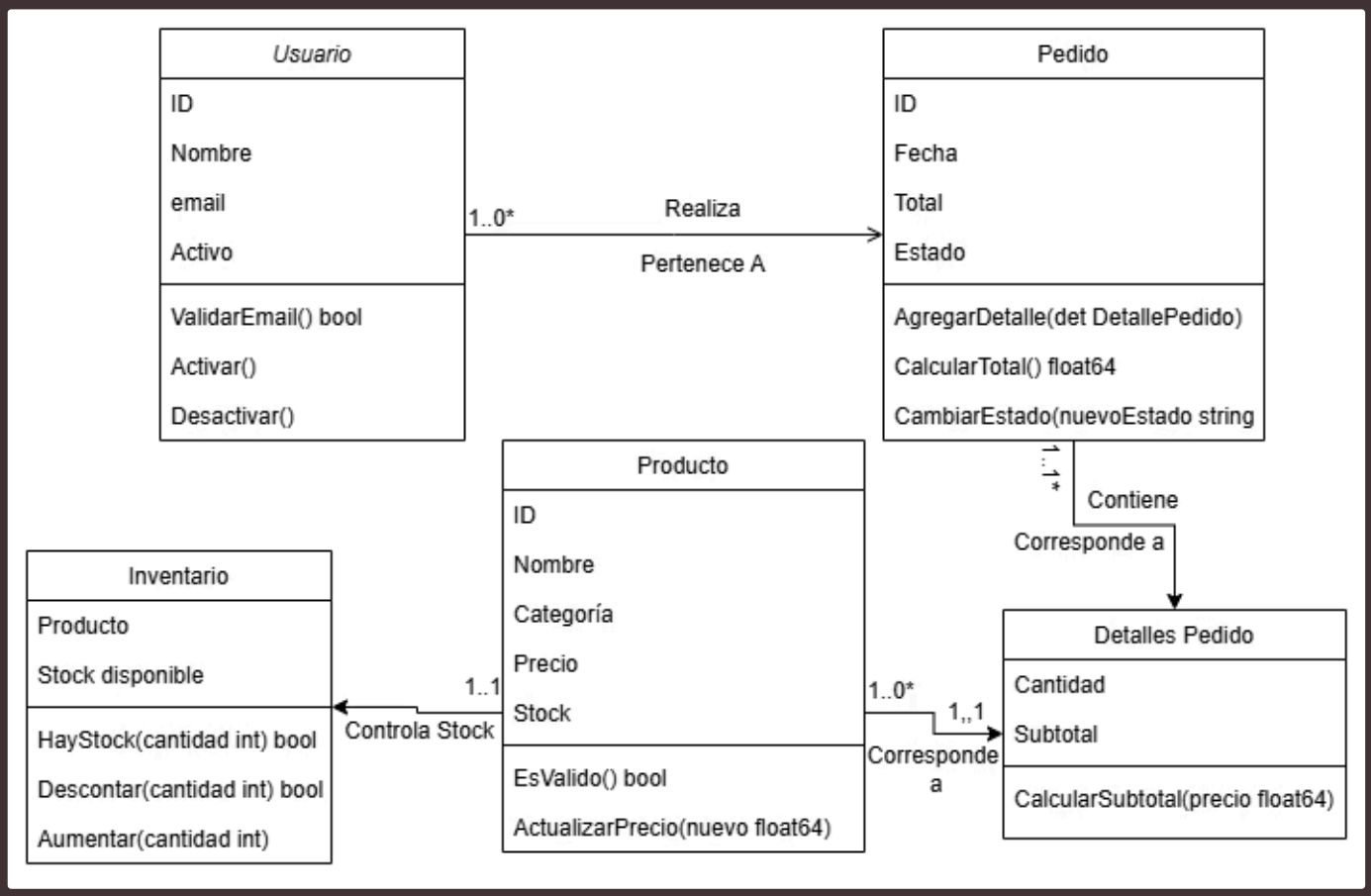
Representa un pedido con: ID, referencia al usuario, fecha, estado y colección de OrderItem. Calcula el total sumando todos los ítems del pedido.

Clase OrderItem

Detalle de productos en un pedido con: referencia al producto, cantidad solicitada y precio unitario al momento de la compra. Permite rastrear historial de precios.

Las relaciones entre clases establecen asociaciones con multiplicidades específicas: un User puede tener múltiples Orders, un Order contiene varios OrderItems, y cada OrderItem se asocia a un único Product. Esta estructura permite mantener integridad referencial y soportar consultas complejas sobre el historial comercial.

Made with GAMMA



Made with GAMMA

Diseño de la Base de Datos

La base de datos MySQL denominada **ecommerce_db** implementa el esquema diseñado en el modelo UML, creando tablas normalizadas que representan cada entidad del sistema. Las relaciones entre tablas se establecen mediante claves primarias y foráneas.

Tabla	Clave Primaria	Claves Foráneas
usuarios	id_usuario	—
productos	id_producto	—
pedidos	id_pedido	id_usuario
pedido_detalles	(id_pedido, id_producto)	id_pedido, id_producto

Tabla usuarios

Almacena información de usuarios con campos: id_usuario (INT, PK), nombre (VARCHAR), email (VARCHAR), contraseña (VARCHAR), fecha_registro (DATETIME)

Tabla productos

Catálogo de productos con: id_producto (INT, PK), nombre (VARCHAR), descripcion (TEXT), precio (DECIMAL), stock (INT), categoria (VARCHAR)

Tabla pedidos

Encabezado de transacciones con: id_pedido (INT, PK), id_usuario (INT, FK), fecha (DATETIME), estado (VARCHAR), total (DECIMAL)

Tabla pedido_detalles

Detalle de ítems con: id_pedido (INT, FK), id_producto (INT, FK), cantidad (INT), precio_unitario (DECIMAL), subtotal (DECIMAL)

Made with GAMMA

Endpoints de la API

El sistema expone una API REST que permite la comunicación entre el frontend HTML y el backend Go. Cada endpoint maneja un tipo específico de operación CRUD sobre las entidades del sistema.

1

POST /api/usuarios

Crea un nuevo usuario en el sistema. Recibe un JSON con nombre, email y contraseña. Retorna el usuario creado con ID asignado.

2

GET /api/usuarios

Consulta todos los usuarios registrados. Retorna un array de objetos JSON con información de cada usuario, excluyendo la contraseña.

3

POST /api/productos

Agrega un nuevo producto al catálogo. Recibe JSON con nombre, descripción, precio y stock. Retorna el producto con ID generado.

4

GET /api/productos

Lista todos los productos disponibles. Retorna array con información completa de cada producto, incluyendo stock actual.

5

POST /api/pedidos

Procesa un nuevo pedido. Recibe JSON con id_usuario y array de ítems (id_producto y cantidad). Calcula totales y actualiza inventario.

6

GET /api/pedidos

Consulta el historial completo de pedidos. Retorna array con información de cada pedido incluyendo usuario, fecha, estado y total.

7

GET /api/resumen

Genera métricas de negocio. Retorna JSON con: ingresos totales, número de pedidos, productos vendidos y análisis de usuarios activos.

Made with GAMMA

Manual de Ejecución y Conclusiones

Manual de Ejecución del Sistema

Restaurar Base de Datos

Ejecute el script SqlDatabase.sql en MySQL para crear ecommerce_db y tablas iniciales con estructura definida.

Configurar Credenciales

Modifique database.go con sus credenciales de MySQL: usuario, contraseña, host (localhost) y nombre de base de datos.

Ejecutar Aplicación

Compile y ejecute con: go run main.go. El servidor iniciará en puerto 8080 y estará listo para recibir conexiones.

Acceder al Sistema

Abrir navegador y navegar a http://localhost:8080 para acceder a la interfaz de gestión del ecommerce.

Conclusiones

Este proyecto demuestra la aplicación práctica de conceptos fundamentales de ingeniería de software: la **arquitectura API REST** permite separar claramente las capas de presentación y lógica de negocio, el **diseño modular en Go** facilita la mantenibilidad y escalabilidad del código, y el **modelado UML** proporciona una visión clara de las entidades y relaciones del sistema.

La implementación de **MySQL como motor de base de datos** garantiza integridad referencial y soporte para operaciones transaccionales complejas. La automatización del cálculo de totales, gestión de inventario y generación de resúmenes comerciales demuestra la capacidad del sistema para reducir tareas manuales y minimizar errores.

Este proyecto cumple con los estándares de documentación técnica requeridos en entornos profesionales, incluyendo diagramas de flujo, casos de uso y clases que facilitan la comprensión del sistema por parte de desarrolladores y evaluadores académicos.

Made with GAMMA